```python
import os
import pickle
import torch
import torch.nn as nn

# Define model directory
base_dir = os.path.dirname(os.path.abspath(__file__))
model_dir = os.path.join(base_dir, 'models')

models = {}

# Load scikit-learn or XGBoost model
with open(os.path.join(model_dir, 'rf_bracket_data.pkl'), 'rb') as f:
    Random_Forest_Model = pickle.load(f)

with open(os.path.join(model_dir, 'xgb_model.pkl'), 'rb') as f:
    XGBoost_Model = pickle.load(f)

# Load Machine Learning Models - PyTorch
# Linear Activation Mode
# Define the same architecture used during training
class LinearModel(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(LinearModel, self).__init__()
        self.linear = nn.Linear(input_dim, output_dim)

    def forward(self, x):
        return self.linear(x)

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Load model weights
model_path = os.path.join("Bracket", "models", "linact_linfea_data.pth")
state_dict = torch.load(model_path, map_location=device, weights_only=True)
```

```python

# Reconstruct model and load weights
input_dim = state_dict['linear.weight'].shape[1]  # infer input dim from weights
linact_linfea_Model = LinearModel(input_dim=input_dim, output_dim=1).to(device)
linact_linfea_Model.load_state_dict(state_dict)
linact_linfea_Model.eval()

# Neural Network Model
# Define the architecture used during training
class MarMad_Neural_Net(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim, activation_fn=nn.ReLU, dropout=0.2):
        super().__init__()
        layers = []
        in_dim = input_dim
        for h_dim in hidden_dims:
            layers.append(nn.Linear(in_dim, h_dim))
            layers.append(activation_fn())
            if dropout > 0:
                layers.append(nn.Dropout(dropout))
            in_dim = h_dim
        layers.append(nn.Linear(in_dim, output_dim))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

# Load weights into the model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_path = os.path.join("Bracket", "models", "NN_linfea_data.pth")
state_dict = torch.load(model_path, map_location=device, weights_only=True)

# Infer input dimension from first layer
input_dim = state_dict['network.0.weight'].shape[1]
hidden_dims = [256, 64, 32, 8]
output_dim = 1
```

```
dropout_rate = 0.2

NN_linfea_Model = MarMad_Neural_Net(input_dim, hidden_dims, output_dim, activation_fn=nn.ReLU, dropout=dropout_rate).to(device)
NN_linfea_Model.load_state_dict(state_dict)
NN_linfea_Model.eval()

# Register all models in a dictionary / Increase flexibility of calling models
models = {
    'rf': Random_Forest_Model,
    'xgb': XGBoost_Model,
    'linact': linact_linfea_Model,
    'nn': NN_linfea_Model
}


```